

OOP

Introduction

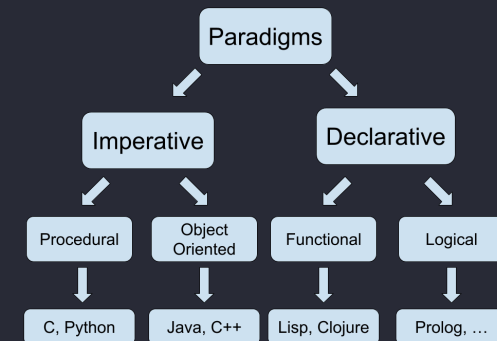
CS-3180

Procedural

- Most of the code we've seen so far
- Mostly 'Procedural'
- Lays out step-by-step instructions
- Functions often simple lay out steps
- This **can** scale, with best practices*

Paradigms

- Paradigms dictate the way we conceptual data
- Specifically, how it flows through your code
- As always, this is just an abstraction
- Over binary, over assembly, over bytecode



OOP

- Object-Oriented Programming
- You are likely somewhat familiar
- UML -> Abstraction on top of OOP
- Rebuild this paradigm through natural abstraction

Button
- xsize
- ysize
- label_text
- interested_listeners
- xposition
- yposition
+ draw()
+ press()
+ register_callback()
+ unregister_callback()

Interaction

- Think about what we can 'use' in our code
- Boils down to two things:

$f(x)$

Interaction

- Think about what we can 'use' in our code
- Boils down to two things:
- Variables: Data (including globals)

$f(x)$

Interaction

- Think about what we can 'use' in our code
- Boils down to two things:
- Variables: Data (including globals)
- Functions: Transform Data

$f(x)$

Objects

- If we are going to abstract beyond this
- It needs to 'black-box' these ideas
- This is naturally what a class does
- Contains data and methods to transform that data
- Let's see why this is useful

```
1 public class Oop {
2     public static void main(String[] args){
3
4         String student1Name = "Alice";
5         String[] student1Classes = {
6             "CS-1181",
7             "CS-3180"
8         };
9
10        String student2Name = "Bob";
11        String[] student2Classes = {
12            "CEG-2350",
13            "CEG-3320"
14        };
15
16        goToClass(student1Name, student1Classes);
17        goToClass(student2Name, student2Classes);
18    }
19
20    public static void goToClass(String name, String[] classes){
21        System.out.print(name + " goes to: ");
22        for(int i = 0; i < classes.length; i++){
23            System.out.print(classes[i] + " ");
24        }
25        System.out.println();
26    }
```

```
1 public class Oop {
2     public static void main(String[] args){
3
4         String student1Name = "Alice";
5         String[] student1Classes = {
6             "CS-1181",
7             "CS-3180"
8         };
9
10        String student2Name = "Bob";
11        String[] student2Classes = {
12            "CEG-2350",
13            "CEG-3320"
14        };
15
16        goToClass(student1Name, student1Classes);
17        goToClass(student2Name, student2Classes);
18    }
19
20    public static void goToClass(String name, String[] classes){
21        System.out.print(name + " goes to: ");
22        for(int i = 0; i < classes.length; i++){
23            System.out.print(classes[i] + " ");
24        }
25        System.out.println();
26    }
```

```
1 public class Oop {
2     public static void main(String[] args){
3
4         String student1Name = "Alice";
5         String[] student1Classes = {
6             "CS-1181",
7             "CS-3180"
8         };
9
10        String student2Name = "Bob";
11        String[] student2Classes = {
12            "CEG-2350",
13            "CEG-3320"
14        };
15
16        goToClass(student1Name, student1Classes);
17        goToClass(student2Name, student2Classes);
18    }
19
20    public static void goToClass(String name, String[] classes){
21        System.out.print(name + " goes to: ");
22        for(int i = 0; i < classes.length; i++){
23            System.out.print(classes[i] + " ");
24        }
25        System.out.println();
26    }
```

```
1 public class Oop {
2     public static void main(String[] args){
3
4         String student1Name = "Alice";
5         String[] student1Classes = {
6             "CS-1181",
7             "CS-3180"
8         };
9
10        String student2Name = "Bob";
11        String[] student2Classes = {
12            "CEG-2350",
13            "CEG-3320"
14        };
15
16        goToClass(student1Name, student1Classes);
17        goToClass(student2Name, student2Classes);
18    }
19
20    public static void goToClass(String name, String[] classes){
21        System.out.print(name + " goes to: ");
22        for(int i = 0; i < classes.length; i++){
23            System.out.print(classes[i] + " ");
24        }
25        System.out.println();
26    }
```

```
1 public class Oop {
2     public static void main(String[] args){
3
4         String student1Name = "Alice";
5         String[] student1Classes = {
6             "CS-1181",
7             "CS-3180"
8         };
9
10        String student2Name = "Bob";
11        String[] student2Classes = {
12            "CEG-2350",
13            "CEG-3320"
14        };
15
16        goToClass(student1Name, student1Classes);
17        goToClass(student2Name, student2Classes);
18    }
19
20    public static void goToClass(String name, String[] classes){
21        System.out.print(name + " goes to: ");
22        for(int i = 0; i < classes.length; i++){
23            System.out.print(classes[i] + " ");
24        }
25        System.out.println();
26    }
```

Issues

- We are just passing data into these functions
- Hope that the caller uses this appropriately
- No way to strictly enforce this

```
1 goToClass(student1Name, student1Classes);  
2 goToClass(student2Name, student2Classes);  
3 // could also do:  
4 goToClass(student2Name, student1Classes);  
5 goToClass(student2Name, student1Classes);
```

Issues

- We are just passing data into these functions
- Hope that the caller uses this appropriately
- No way to strictly enforce this

```
1 goToClass(student1Name, student1Classes);  
2 goToClass(student2Name, student2Classes);  
3 // could also do:  
4 goToClass(student2Name, student1Classes);  
5 goToClass(student2Name, student1Classes);
```

Issues

- We are just passing data into these functions
- Hope that the caller uses this appropriately
- No way to strictly enforce this

```
1 goToClass(student1Name, student1Classes);  
2 goToClass(student2Name, student2Classes);  
3 // could also do:  
4 goToClass(student2Name, student1Classes);  
5 goToClass(student2Name, student1Classes);
```

Issues

- We are just passing data into these functions
- Hope that the caller uses this appropriately
- No way to strictly enforce this

```
1 goToClass(student1Name, student1Classes);  
2 goToClass(student2Name, student2Classes);  
3 // could also do:  
4 goToClass(student2Name, student1Classes);  
5 goToClass(student2Name, student1Classes);
```

Coupling

- We would like a way to tightly couple data
- Student is composed of:
- Their Name
- Their Classes
- We don't want to mismatch these
- Let's re-write this with a class

OOP Vocab

- Classes: Blueprints for data
- Objects: Instance of that coupled data
- Fields: Data specific to the instance
- Methods: Functions that accept object data
- Constructor: Special method to initialize an object

Refactor with Classes

```
1 class Student {
2     // fields
3     String name;
4     String[] classes;
5     // constructor
6     public Student(String name, String[] classes) {
7         this.name = name;
8         this.classes = classes;
9     }
10    // method
11    public void goToClass(){
12        System.out.print(name + " goes to: ");
13        for(int i = 0; i < classes.length; i++){
14            System.out.print(classes[i] + " ");
15        }
16        System.out.println();
17    }
18 }
```

Refactor with Classes

```
1 class Student {
2     // fields
3     String name;
4     String[] classes;
5     // constructor
6     public Student(String name, String[] classes) {
7         this.name = name;
8         this.classes = classes;
9     }
10    // method
11    public void goToClass(){
12        System.out.print(name + " goes to: ");
13        for(int i = 0; i < classes.length; i++){
14            System.out.print(classes[i] + " ");
15        }
16        System.out.println();
17    }
18 }
```

Refactor with Classes

```
1 class Student {
2     // fields
3     String name;
4     String[] classes;
5     // constructor
6     public Student(String name, String[] classes) {
7         this.name = name;
8         this.classes = classes;
9     }
10    // method
11    public void goToClass(){
12        System.out.print(name + " goes to: ");
13        for(int i = 0; i < classes.length; i++){
14            System.out.print(classes[i] + " ");
15        }
16        System.out.println();
17    }
18 }
```

Refactor with Classes

```
1 class Student {
2     // fields
3     String name;
4     String[] classes;
5     // constructor
6     public Student(String name, String[] classes) {
7         this.name = name;
8         this.classes = classes;
9     }
10    // method
11    public void goToClass(){
12        System.out.print(name + " goes to: ");
13        for(int i = 0; i < classes.length; i++){
14            System.out.print(classes[i] + " ");
15        }
16        System.out.println();
17    }
18 }
```

Refactor with Classes

```
~ // constructor
6   public Student(String name, String[] classes) {
7       this.name = name;
8       this.classes = classes;
9   }
10  // method
11  public void goToClass(){
12      System.out.print(name + " goes to: ");
13      for(int i = 0; i < classes.length; i++){
14          System.out.print(classes[i] + " ");
15      }
16      System.out.println();
17  }
18 }
19
20 class Oop {
21     public static void main(String[] args) {
22         Student s1 = new Student("Alice", new String[] {
```

Refactor with Classes

```
17     }  
18 }  
19  
20 class Oop {  
21     public static void main(String[] args) {  
22         Student s1 = new Student("Alice", new String[]{  
23             "CS-3180", "CS-1181"  
24         });  
25  
26         Student s2 = new Student("Bob", new String[]{  
27             "CEG-2350",  
28             "CEG-3320"  
29         });  
30  
31         s1.goToClass();  
32         s2.goToClass();  
33     }  
34 }
```

Refactor with Classes

```
19
20 class Oop {
21     public static void main(String[] args) {
22         Student s1 = new Student("Alice", new String[]{
23             "CS-3180", "CS-1181"
24         });
25
26         Student s2 = new Student("Bob", new String[]{
27             "CEG-2350",
28             "CEG-3320"
29         });
30
31         s1.goToClass();
32         s2.goToClass();
33     }
34 }
```

Refactor with Classes

```
1 class Student {
2     // fields
3     String name;
4     String[] classes;
5     // constructor
6     public Student(String name, String[] classes) {
7         this.name = name;
8         this.classes = classes;
9     }
10    // method
11    public void goToClass(){
12        System.out.print(name + " goes to: ");
13        for(int i = 0; i < classes.length; i++){
14            System.out.print(classes[i] + " ");
15        }
16        System.out.println();
17    }
18 }
```

Dead Horse

- You likely already know most of this
- Set up: A contrast of OOP implementations
- Java specificities:

Dead Horse

- You likely already know most of this
- Set up: A contrast of OOP implementations
- Java specificities:
- 'this' keyword: implicit parameter of the particular instance

Dead Horse

- You likely already know most of this
- Set up: A contrast of OOP implementations
- Java specificities:
- 'this' keyword: implicit parameter of the particular instance
- Everything is in a class: Even non-OOP code

Dead Horse

- You likely already know most of this
- Set up: A contrast of OOP implementations
- Java specificities:
- 'this' keyword: implicit parameter of the particular instance
- Everything is in a class: Even non-OOP code
- Constructor: Special method with it's own name

Dead Horse

- You likely already know most of this
- Set up: A contrast of OOP implementations
- Java specificities:
- 'this' keyword: implicit parameter of the particular instance
- Everything is in a class: Even non-OOP code
- Constructor: Special method with it's own name
- Fields: Explicitly named

Python OOP

- Let's take look at how python handles this same situation
- Java has basically defined OOP
- Many decisions are arbitrary
- With respect to those nuances

Python Classes

```
1 class Student:
2     def __init__(self, name, classes):
3         self.name = name
4         self.classes = classes
5
6     def go_to_class(self):
7         # class keyword reserved
8         print(self.name + " goes to: ")
9         for course in self.classes:
10             print(course)
11
12 s1= Student("Alice", [
13     "CS-1181",
14     "CS-3180"
15 ])
16
```

Python Classes

```
1 class Student:
2     def __init__(self, name, classes):
3         self.name = name
4         self.classes = classes
5
6     def go_to_class(self):
7         # class keyword reserved
8         print(self.name + " goes to: ")
9         for course in self.classes:
10             print(course)
11
12 s1= Student("Alice", [
13     "CS-1181",
14     "CS-3180"
15 ])
16
```

Python Classes

```
1 class Student:
2     def __init__(self, name, classes):
3         self.name = name
4         self.classes = classes
5
6     def go_to_class(self):
7         # class keyword reserved
8         print(self.name + " goes to: ")
9         for course in self.classes:
10             print(course)
11
12 s1= Student("Alice", [
13     "CS-1181",
14     "CS-3180"
15 ])
16
```

Python Classes

```
1 class Student:
2     def __init__(self, name, classes):
3         self.name = name
4         self.classes = classes
5
6     def go_to_class(self):
7         # class keyword reserved
8         print(self.name + " goes to: ")
9         for course in self.classes:
10             print(course)
11
12 s1= Student("Alice", [
13     "CS-1181",
14     "CS-3180"
15 ])
16
```

Python Classes

```
8         print(self.name + " goes to: ")
9         for course in self.classes:
10             print(course)
11
12 s1= Student("Alice", [
13     "CS-1181",
14     "CS-3180"
15 ])
16
17 s2= Student("Bob", [
18     "CEG-2350",
19     "CEG-3320"
20 ])
21
22 s1.go_to_class()
23 s2.go_to_class()
```

Python Classes

```
8         print(self.name + " goes to: ")
9         for course in self.classes:
10             print(course)
11
12 s1= Student("Alice", [
13     "CS-1181",
14     "CS-3180"
15 ])
16
17 s2= Student("Bob", [
18     "CEG-2350",
19     "CEG-3320"
20 ])
21
22 s1.go_to_class()
23 s2.go_to_class()
```

Python Classes

```
1 class Student:
2     def __init__(self, name, classes):
3         self.name = name
4         self.classes = classes
5
6     def go_to_class(self):
7         # class keyword reserved
8         print(self.name + " goes to: ")
9         for course in self.classes:
10             print(course)
11
12 s1= Student("Alice", [
13     "CS-1181",
14     "CS-3180"
15 ])
16
```

Python Trickiness

- Classes are optional
- `__init__` used for a constructor
- Dynamically add fields, without explicit declaration
- 'self' replaces 'this'
- Explicitly stated in declaration
- Implicitly passed

```
# s1 IS the self parameter  
s1.go_to_class()
```

'Dunder' Methods

- Double Underscore = 'Dunder'
- Python treats these as special methods
- Usually called implicitly
- There is a lot of these you can override
- E.g. `__add__` gets called when you use `+` on two classes

examples

```
1 class Number:
2     # gets called when instantiated
3     def __init__(self, value):
4         self.value = value
5
6     # gets called when + is used
7     def __add__(self, o):
8         return Number(self.value + o.value)
9
10    # string representation
11    def __repr__(self):
12        return str(self.value)
13
14
15 n1 = Number(2)
16 n2 = Number(3)
```

examples

```
1 class Number:
2     # gets called when instantiated
3     def __init__(self, value):
4         self.value = value
5
6     # gets called when + is used
7     def __add__(self, o):
8         return Number(self.value + o.value)
9
10    # string representation
11    def __repr__(self):
12        return str(self.value)
13
14
15 n1 = Number(2)
16 n2 = Number(3)
```

examples

```
1 class Number:
2     # gets called when instantiated
3     def __init__(self, value):
4         self.value = value
5
6     # gets called when + is used
7     def __add__(self, o):
8         return Number(self.value + o.value)
9
10    # string representation
11    def __repr__(self):
12        return str(self.value)
13
14
15 n1 = Number(2)
16 n2 = Number(3)
```

examples

```
3     def __init__(self, value):
4         self.value = value
5
6     # gets called when + is used
7     def __add__(self, o):
8         return Number(self.value + o.value)
9
10    # string representation
11    def __repr__(self):
12        return str(self.value)
13
14
15    n1 = Number(2)
16    n2 = Number(3)
17
18    result = n1 + n2
19    print(result)
```

examples

```
4         self.value = value
5
6         # gets called when + is used
7         def __add__(self, o):
8             return Number(self.value + o.value)
9
10        # string representation
11        def __repr__(self):
12            return str(self.value)
13
14
15 n1 = Number(2)
16 n2 = Number(3)
17
18 result = n1 + n2
19 print(result)
```

examples

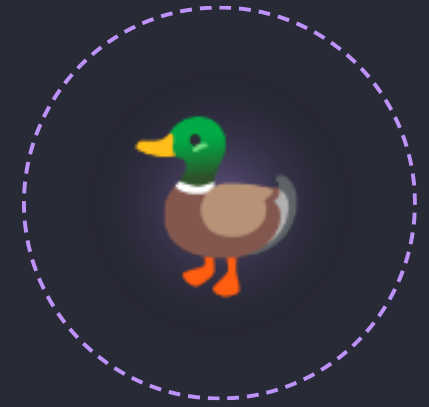
```
1 class Number:
2     # gets called when instantiated
3     def __init__(self, value):
4         self.value = value
5
6     # gets called when + is used
7     def __add__(self, o):
8         return Number(self.value + o.value)
9
10    # string representation
11    def __repr__(self):
12        return str(self.value)
13
14
15 n1 = Number(2)
16 n2 = Number(3)
```

Dynamic Typing Classes

- Java gets to check that a variable is a specific type
- Enforce this at compile time
- Python can't really do the same
- With 'primitives': checks your usage (+, *, etc) is valid at runtime*
- With classes, this is done at method-access time
- (Secretly, primitives are really classes)

Duck Typing

- "If it looks like a duck and quacks like a duck"
- It's probably a duck
- Call the method
- Object type is irrelevant
- Just needs the appropriate method
- Let's see what this means in practice



Canonical Example

- Python gives a free no-arg constructor

```
1 class Duck:
2     def quack(self):
3         print("Duck Quack!")
4
5 class Person:
6     def quack(self):
7         print("*Human like imitation of a quack*")
8
9 class Dog:
10    def bark(self):
11        print("Woof!")
12
13
14 # will check if this method exists at runtime, then call it
15 # since the animal parameter doesn't have a type
16 def make_it_quack(animal):
```

Canonical Example

- Python gives a free no-arg constructor

```
1 class Duck:
2     def quack(self):
3         print("Duck Quack!")
4
5 class Person:
6     def quack(self):
7         print("*Human like imitation of a quack*")
8
9 class Dog:
10    def bark(self):
11        print("Woof!")
12
13
14 # will check if this method exists at runtime, then call it
15 # since the animal parameter doesn't have a type
16 def make_it_quack(animal):
```

Canonical Example

- Python gives a free no-arg constructor

```
1 class Duck:
2     def quack(self):
3         print("Duck Quack!")
4
5 class Person:
6     def quack(self):
7         print("*Human like imitation of a quack*")
8
9 class Dog:
10    def bark(self):
11        print("Woof!")
12
13
14 # will check if this method exists at runtime, then call it
15 # since the animal parameter doesn't have a type
16 def make_it_quack(animal):
```

Canonical Example

- Python gives a free no-arg constructor

```
2     def quack(self):
3         print("Duck Quack!")
4
5 class Person:
6     def quack(self):
7         print("*Human like imitation of a quack*")
8
9 class Dog:
10    def bark(self):
11        print("Woof!")
12
13
14 # will check if this method exists at runtime, then call it
15 # since the animal parameter doesn't have a type
16 def make_it_quack(animal):
17     animal.quack()
18
```

Canonical Example

- Python gives a free no-arg constructor

```
7         print("Human-like imitation of a quack")
8
9 class Dog:
10     def bark(self):
11         print("Woof!")
12
13
14 # will check if this method exists at runtime, then call it
15 # since the animal parameter doesn't have a type
16 def make_it_quack(animal):
17     animal.quack()
18
19 # directly instantiate each type
20 make_it_quack(Duck())
21 # person has a duck method, so this works
22 make_it_quack(Person())
23
24 # AttributeError is raised
```

Canonical Example

- Python gives a free no-arg constructor

```
12
13
14 # will check if this method exists at runtime, then call it
15 # since the animal parameter doesn't have a type
16 def make_it_quack(animal):
17     animal.quack()
18
19 # directly instantiate each type
20 make_it_quack(Duck())
21 # person has a duck method, so this works
22 make_it_quack(Person())
23
24 # AttributeError is raised
25 # attempting to access an attribute that doesn't exist
26 make_it_quack(Dog())
27
```

Duck Typing

- This is *mostly* unique to dynamic, interpreted languages
- Javascript, Ruby, PHP have similar systems
- Was clearly not created with OOP in mind
- Just supports OOP features

Duck Typing

- This is *mostly* unique to dynamic, interpreted languages
- Javascript, Ruby, PHP have similar systems
- Was clearly not created with OOP in mind
- Just supports OOP features
- Would you call python an object-oriented language?

Encapsulation

- Abstraction is about hiding data
- Only expose the necessary parts of system
- We don't want the users to have unfettered access to our data
- That's part of why we made a class to begin with
- Control the state
- Encode rules into your type system

- Java: private/public/protected keywords

```
1 public class Account {  
2  
3     // prevent people from accessing via account.balance  
4     private int balance = 1000;  
5  
6  
7     // allow anyone to create an account  
8     public Account(int startingBalance){  
9         this.balance = startingBalance;  
10    }  
11  
12    // public access function  
13    // allow the world to see  
14    public int getBalance(){  
15        return this.balance;  
16    }
```

- Java: private/public/protected keywords

```
1 public class Account {  
2  
3     // prevent people from accessing via account.balance  
4     private int balance = 1000;  
5  
6  
7     // allow anyone to create an account  
8     public Account(int startingBalance){  
9         this.balance = startingBalance;  
10    }  
11  
12    // public access function  
13    // allow the world to see  
14    public int getBalance(){  
15        return this.balance;  
16    }
```

- Java: private/public/protected keywords

```
1 public class Account {
2
3     // prevent people from accessing via account.balance
4     private int balance = 1000;
5
6
7     // allow anyone to create an account
8     public Account(int startingBalance){
9         this.balance = startingBalance;
10    }
11
12    // public access function
13    // allow the world to see
14    public int getBalance(){
15        return this.balance;
16    }
17 }
```

- Java: private/public/protected keywords

```
6
7    // allow anyone to create an account
8    public Account(int startingBalance){
9        this.balance = startingBalance;
10   }
11
12   // public access function
13   // allow the world to see
14   public int getBalance(){
15       return this.balance;
16   }
17
18   // private mutator function
19   // prevent world from editing
20   private void setBalance(int newBalance) {
21       if(this.balance - newBalance < 0){
22           throw new RuntimeException();
```

- Java: private/public/protected keywords

```
14     public int getBalance(){
15         return this.balance;
16     }
17
18     // private mutator function
19     // prevent world from editing
20     private void setBalance(int newBalance) {
21         if(this.balance - newBalance < 0){
22             throw new RuntimeException();
23         }
24
25         this.balance = newBalance;
26     }
27
28     // enforce the caller to use a method with
29     // other parameters/guards on it
30     public void processTransaction(Transaction t) {
```

- Java: private/public/protected keywords

```
20     private void setBalance(int newBalance) {
21         if(this.balance - newBalance < 0){
22             throw new RuntimeException();
23         }
24
25         this.balance = newBalance;
26     }
27
28     // enforce the caller to use a method with
29     // other parameters/guards on it
30     public void processTransaction(Transaction t) {
31         System.out.println("Processing transaction...");
32         setBalance(t.amount);
33     }
34
35 }
```

- Java: private/public/protected keywords

```
1 public class Account {  
2  
3     // prevent people from accessing via account.balance  
4     private int balance = 1000;  
5  
6  
7     // allow anyone to create an account  
8     public Account(int startingBalance){  
9         this.balance = startingBalance;  
10    }  
11  
12    // public access function  
13    // allow the world to see  
14    public int getBalance(){  
15        return this.balance;  
16    }
```

- Keywords allow us to restrict the access
- Accessing a private member: Code does not compile

```
class Main {  
    public static void main(String[] args) {  
  
        Account a = new Account(500);  
        // allowed to access public methods  
        a.processTransaction(new Transaction(100));  
  
        // does not compile  
        a.balance = 400;  
    }  
}
```

Python Encapsulation

- Python does NOT have any similar protection
- Enforced via convention:
- Use an underscore for private members
- You can still change them

No Safe Guards

```
1 class Account:
2
3     def __init__(self, startingBalance):
4         # single underscore for fields
5         self._balance = startingBalance
6
7     def get_balance(self):
8         return self._balance
9
10    # two underscores for method
11    def __set_balance(self, newBalance):
12        if self._balance - newBalance < 0:
13            raise RuntimeError()
14        self._balance = newBalance
15
16    def process_transaction(self, amount):
```

No Safe Guards

```
1 class Account:
2
3     def __init__(self, startingBalance):
4         # single underscore for fields
5         self._balance = startingBalance
6
7     def get_balance(self):
8         return self._balance
9
10    # two underscores for method
11    def __set_balance(self, newBalance):
12        if self._balance - newBalance < 0:
13            raise RuntimeError()
14        self._balance = newBalance
15
16    def process_transaction(self, amount):
```

No Safe Guards

```
4         # single underscore for fields
5         self._balance = startingBalance
6
7     def get_balance(self):
8         return self._balance
9
10    # two underscores for method
11    def __set_balance(self, newBalance):
12        if self._balance - newBalance < 0:
13            raise RuntimeError()
14        self._balance = newBalance
15
16    def process_transaction(self, amount):
17        print("Processing transaction...")
18        self.__set_balance(amount)
19
20
```

No Safe Guards

```
11     def __set_balance(self, newBalance):
12         if self._balance - newBalance < 0:
13             raise RuntimeError()
14         self._balance = newBalance
15
16     def process_transaction(self, amount):
17         print("Processing transaction...")
18         self.__set_balance(amount)
19
20
21 a = Account(500)
22 a.process_transaction(100)
23
24 a._balance = -200
25 print(a._balance)
26
```

No Safe Guards

```
11     def __set_balance(self, newBalance):
12         if self._balance - newBalance < 0:
13             raise RuntimeError()
14         self._balance = newBalance
15
16     def process_transaction(self, amount):
17         print("Processing transaction...")
18         self.__set_balance(amount)
19
20
21 a = Account(500)
22 a.process_transaction(100)
23
24 a._balance = -200
25 print(a._balance)
26
```

No Safe Guards

```
1 class Account:
2
3     def __init__(self, startingBalance):
4         # single underscore for fields
5         self._balance = startingBalance
6
7     def get_balance(self):
8         return self._balance
9
10    # two underscores for method
11    def __set_balance(self, newBalance):
12        if self._balance - newBalance < 0:
13            raise RuntimeError()
14        self._balance = newBalance
15
16    def process_transaction(self, amount):
```

Summary

- Object-Oriented Programming
- Natural abstraction upon variables + functions
- Different Languages approaches
- Encapsulation of data
- Desired for scalable system
- OOP implementations differ

Questions?